

DBMS Experiential Learning – Level 3 Project

Project Requirements

1. Project Title

- Each team must identify a **unique project title within the section**.
- The project should address a meaningful real-world problem and should not be a simple CRUD application.

2. Team Formation

- Projects should be completed as 2 member teams as approved by the course instructor.
- Each team member must have clearly defined responsibilities and contribute to both database and application development.

3. Database Requirement

- The project must demonstrate **practical database design and implementation**.
- Students has to use:

Option 1: SQL + NoSQL

- **SQL Database (Mandatory):** MySQL or PostgreSQL
- **NoSQL Database:** MongoDB, Neo4j, etc.

Option 2: SQL + Vector Database

- **SQL Database (Mandatory):** MySQL or PostgreSQL
- **Vector Database:** PostgreSQL with pgvector, Qdrant, Pinecone, Chroma, etc.
- A project may use two **(Minimum) or more database paradigms**, depending on its requirements.

4. Database Design

- For SQL-based components, students must prepare an **ER diagram** and convert it into a **relational schema**.
- The database design must identify:
 - Entities/tables
 - Attributes
 - Primary keys
 - Foreign keys
 - Relationships
 - Appropriate constraints
- For NoSQL/vector components, students must document the **data model, collections/documents, indexes, embeddings, metadata, and retrieval strategy**, as applicable.

5. SQL Implementation

- Students using SQL must create and execute appropriate **DDL statements**.

- The project should demonstrate suitable use of:
 - CREATE
 - ALTER
 - DROP
 - Primary and foreign keys
 - Constraints
 - Indexes
 - Views, where appropriate

6. Database Operations

- The application must demonstrate meaningful database operations using **DML**:
 - INSERT
 - UPDATE
 - DELETE
 - SELECT
- Appropriate queries involving **JOIN, GROUP BY, HAVING, ORDER BY, aggregate functions, subqueries, etc.** should be demonstrated wherever applicable.

7. Advanced Database Features

- Each project should demonstrate at least **four advanced database concepts**, such as:
 - Transactions
 - Views
 - Stored procedures/functions
 - Triggers
 - Indexing
 - Full-text search
 - NoSQL aggregation
 - Vector similarity search

8. Vector Database / Semantic Search — Optional but Recommended

- Projects involving unstructured or text-heavy data are encouraged to implement **vector embeddings and similarity search**.
- Students should explain:
 - What data is converted into embeddings
 - Which embedding approach/model is used
 - How vectors are stored
 - Similarity metric used
 - How relevant results are retrieved

9. Web Application Interface

- The project must provide a functional **web-based interface using Next.js**.
- The interface should allow users to interact with the implemented database functionality.
- The application should include appropriate forms, tables, search, filtering, dashboards, or other UI components relevant to the project.

10. Database–Application Integration

- The Next.js application must communicate with the database through an appropriate backend/API layer.
- Database credentials and sensitive configuration must **not be hard-coded** in the source code.
- Environment variables should be used for database configuration.

11. Authentication and Authorization

- Projects should implement appropriate **user authentication** where relevant.
- Different roles such as **Admin, Staff, Student, Customer, etc.** should be considered when the problem domain requires role-based access(Minimum 2 roles)

12. Data Validation and Integrity

- Input validation must be implemented at the database level.
- The project should demonstrate how **data consistency and integrity** are maintained.

13. Security

- Students must follow basic database and web security practices.
- Passwords and API/database credentials must not be stored in plain text.

14. Project Documentation

Each team must maintain documentation containing:

- Problem statement
- Objectives
- Scope
- Functional requirements
- ER diagram
- Relational schema
- Database choice and justification
- Data dictionary/schema description
- SQL/NoSQL/vector database implementation
- Important queries
- System architecture
- Application screenshots
- Testing results
- Conclusion and future enhancements

15. Git/GitHub

- Each team should maintain the project source code using **Git/GitHub**.
- Commits should demonstrate **individual contributions** of team members.
- The repository should contain a suitable **README** with setup and execution instructions.

16. Evaluation / Demonstration

The final demonstration should include:

- Problem statement and motivation
- ER diagram and database design
- Relational schema, where applicable

- Database implementation
- Important queries/operations
- Next.js interface
- Database integration
- Advanced database features
- Test cases
- Individual contribution of team members

17. Originality

- Projects should demonstrate **original design and implementation**.
- Directly copying an existing project, tutorial, or repository is not permitted.
- Teams working on similar domains must have substantially different **problem statements, features, database designs, and implementations**.

18. Level 3 Expectation

- The project should go beyond basic **Create–Read–Update–Delete (CRUD)** functionality.
- Students are expected to demonstrate a clear connection between **database concepts, database design, implementation, and a functional web application**.
- The choice of SQL, NoSQL, and/or vector database should be **justified based on the nature of the data and application requirements**, rather than being included merely for technology demonstration.

Suggested minimum technology stack

Frontend: Next.js

Backend/API: Next.js API routes / Route Handlers or an appropriate backend service

Database: SQL with NoSQL / Vector Database

Version Control: Git + GitHub

“Database technology must be selected based on the problem requirements, and students must justify why SQL, NoSQL, or a vector database is appropriate for each type of data.”